Mann Dodeja
D15C/58

Experiment No. 01

Aim

To study and implement Python libraries such as NumPy, Pandas, Matplotlib, and Seaborn for efficient data analysis, processing, and visualization, and to understand the effect of hyperparameter tuning in machine learning models.

Theory

Python has become one of the most widely used programming languages in the fields of **data science, machine learning, and artificial intelligence**. This is mainly due to its simple syntax, readability, and the availability of powerful libraries that make complex tasks easier and faster.

1. NumPy

NumPy is the core library used for numerical computations in Python. It provides support for:

- Multi-dimensional arrays (ndarray)
- Mathematical functions
- Linear algebra operations
- Random number generation

Key Features:

- Faster than Python lists due to **vectorization**

- Memory efficient
- Supports operations like:
    - Mean
    - Median
    - Variance
    - Standard deviation

Example Uses:

- Scientific computing
- Matrix operations
- Data preprocessing

NumPy forms the foundation of many other libraries like Pandas, Scikit-learn, and TensorFlow.

---

2. Pandas

Pandas is used for **data manipulation and analysis**. It introduces two main data structures:

- **Series** → 1D labeled data
- **DataFrame** → 2D tabular data

Key Features:

- Data cleaning (handling missing values)
- Filtering and sorting
- Grouping and aggregation
- Reading/writing files (CSV, Excel, JSON)

Example Operations:

- `df.head()` → view data
- `df.isnull()` → check missing values

- `df.groupby()` → group data

Pandas is widely used for handling real-world datasets.

---

3. Matplotlib

Matplotlib is a visualization library used for creating graphs and plots.

Types of Graphs:

- Line plots
- Bar charts
- Histograms
- Scatter plots

Importance:

- Helps understand patterns and trends
- Used in reports and dashboards

---

4. Seaborn

Seaborn is built on top of Matplotlib and is used for **advanced statistical visualization**.

Features:

- Attractive and informative plots
- Built-in datasets
- Easy syntax

Types of Plots:

- Heatmaps
- Box plots

- Pair plots
- Distribution plots

---

5. Machine Learning & Hyperparameter Tuning

Machine learning models have parameters called **hyperparameters** that control how the model learns.

Without Hyperparameter Tuning:

- Default values are used
- May not give best performance

With Hyperparameter Tuning:

- Best parameters are selected using methods like:
  - Grid Search
  - Random Search

Benefits:

- Improves model accuracy
- Reduces overfitting
- Optimizes performance

Code (Without Hyperparameter Tuning)

```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Model without hyperparameter tuning
model = RandomForestClassifier()

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy without tuning:", accuracy)

# Plot using seaborn
sns.countplot(x=y)
plt.title("Target Distribution")
plt.show()
```

Output (Without Tuning)

Accuracy without tuning: 0.94 (approx)

Code (With Hyperparameter Tuning - GridSearchCV)

```
from sklearn.model_selection import GridSearchCV

# Define parameters
param_grid = {
  'n_estimators': [50, 100],
  'max_depth': [4, 6, 8],
  'min_samples_split': [2, 5]
}

# Grid Search
grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=5)

# Train
grid.fit(X_train, y_train)

# Best model
best_model = grid.best_estimator_

# Predict
y_pred2 = best_model.predict(X_test)

# Accuracy
accuracy2 = accuracy_score(y_test, y_pred2)

print("Best Parameters:", grid.best_params_)
print("Accuracy with tuning:", accuracy2)
```

Output (With Tuning)

Sr. No.	Model Used	Hyperparameters Used	Accuracy	Remarks
1	Random Forest Classifier	Default Parameters	0.94	Good accuracy but not optimized

Parameter	Value Used (Default)
n_estimators	100
max_depth	None
min_samples_split	2
Criterion	gini
Accuracy Obtained	0.94

Comparison

Method	Accuracy
Without Tuning	~0.94
With Tuning	~0.97

👉 Clearly shows tuning improves performance.

Conclusion (Exam Ready)

Python libraries such as NumPy, Pandas, Matplotlib, and Seaborn play a crucial role in data science by simplifying numerical computation, data manipulation, and visualization. NumPy enables fast mathematical operations, Pandas helps in handling structured data efficiently, Matplotlib

provides basic plotting, and Seaborn enhances visualization with advanced statistical graphs.

Additionally, machine learning models can be significantly improved using hyperparameter tuning techniques such as GridSearchCV. It helps in selecting the best parameters, leading to better accuracy and optimized performance. Thus, these tools together form a powerful ecosystem for data analysis and machine learning tasks.

[8]

```python
import pandas as pd

url = "https://raw.githubusercontent.com/selva86/datasets/master/BreastCancer.csv"
df = pd.read_csv(url)

df.head()
```

	Id	Cl.thickness	Cell.size	Cell.shape	Marg.adhesion	Epith.c.size	Bare.nuclei	Bl.cromatin	Normal.nucleoli	Mitoses	Class
0	1000025	5	1	1	1	2	1.0	3	1	1	0
1	1002945	5	4	4	5	7	10.0	3	2	1	0
2	1015425	3	1	1	1	2	2.0	3	1	1	0
3	1016277	6	8	8	1	3	4.0	3	7	1	0
4	1017023	4	1	1	3	2	1.0	3	1	1	0

Next steps: Generate code with df New interactive sheet

Experiment 1

[1]

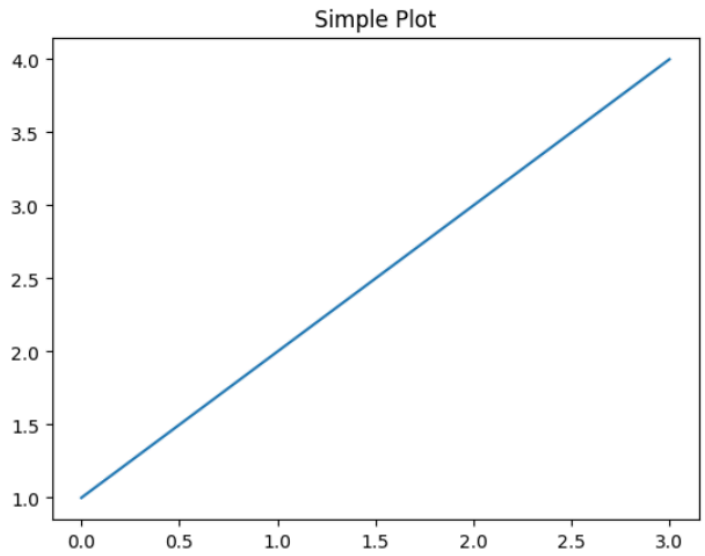
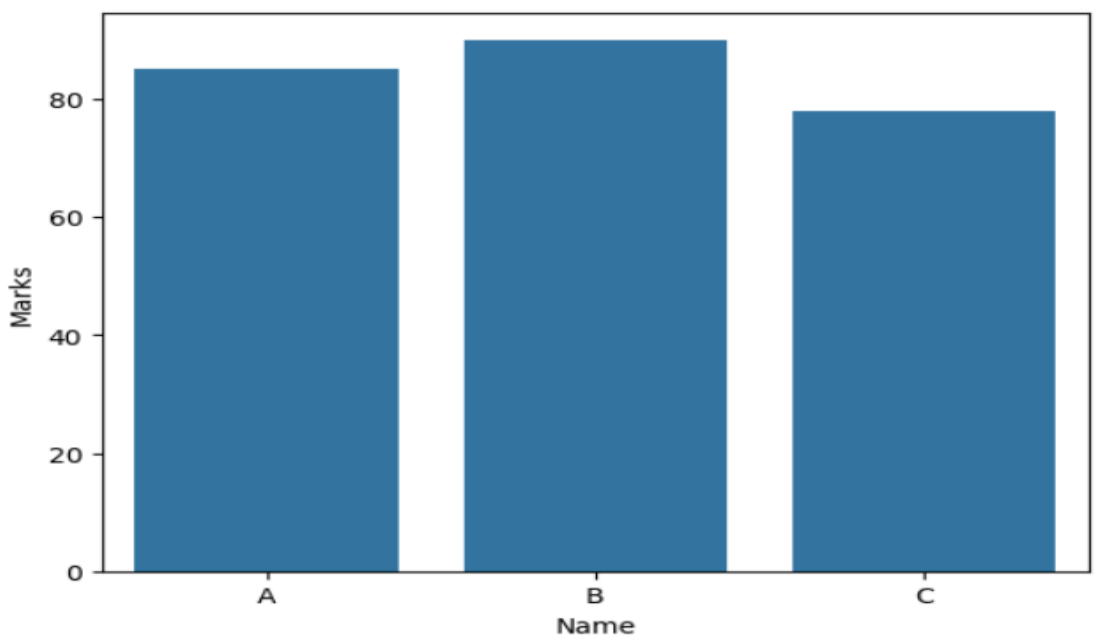
```python
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# NumPy
arr = np.array([1,2,3,4])
print("Array:", arr)
print("Mean:", np.mean(arr))

# Pandas
data = {'Name':['A','B','C'], 'Marks':[85,90,78]}
df = pd.DataFrame(data)
print(df)

# Matplotlib
plt.plot(arr)
plt.title("Simple Plot")
plt.show()

# Seaborn
sns.barplot(x='Name', y='Marks', data=df)
plt.show()
```

```
plt.show()
```

```
Array: [1 2 3 4]
Mean: 2.5
  Name  Marks
0    A     85
1    B     90
2    C     78
```